

```

import numpy as np

import pandas as pd

dataset=pd.read_csv("file.csv")

concepts=np.array(dataset.iloc[:, :-1])

target=np.array(dataset.iloc[:, -1])

def learn(concepts,target):

    s=concepts[0].copy()

    g=[['?' for i in range(len(s))]

    for i in range(len(s))

        for i,h in enumerate(concepts):

            if target[i]=='yes':

                for x in range(len(s)):

                    if s[x]!=h[x]:

                        s[x]='?'

                        g[x][x]='?'

            else:

                for x in range(len(s)):

                    if s[x]!=h[x]:

                        g[x][x]=s[x]

                    else:

                        g[x][x]='?'

    print(f"\nstep {i+1}")

    print("specific hypo:",s)

    print("general hypo:",g)

```

```
        g=[x for x in g if x!='?'] *len(s)]

    return s,g

S,G=learn(concepts,target)

print("Specific Hypothesis",S)

print("General Hypothesis",G)
```

```
import numpy as np
```

```
X = np.array([[2,9],[3,6],[1,5]])

y = np.array([[89],[96],[92]])
```

```
# Normalize data
```

```
X = X / np.max(X)
```

```
y = y / 100
```

```
# Sigmoid function
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
# Derivative of sigmoid
```

```
def sigmoid_prime(x):
```

```
    return x * (1 - x)
```

```

# Initialize weights

w1 = np.random.rand(2,3)

w2 = np.random.rand(3,1)


# Output before training

hidden = sigmoid(np.dot(X, w1))

output = sigmoid(np.dot(hidden, w2))


print("Input:\n", X)

print("Predicted Output:\n", output)

print("Loss Before Training:\n",
      np.mean(np.square(y - output)))


# Training

for i in range(1000):

    hidden = sigmoid(np.dot(X, w1))

    output = sigmoid(np.dot(hidden, w2))

    error = y - output

    d_output = error * sigmoid_prime(output)

    d_hidden = np.dot(d_output, w2.T) * sigmoid_prime(hidden)

    w2 += np.dot(hidden.T, d_output)

    w1 += np.dot(X.T, d_hidden)


# Output after training

```

```
hidden = sigmoid(np.dot(X, w1))
output = sigmoid(np.dot(hidden, w2))

print("Predicted Output:\n", output)
print("Loss After Training:\n",
      np.mean(np.square(y - output)))
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
import numpy as np
from sklearn.preprocessing import LabelEncoder
import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
dataset=pd.read_csv('tennis.csv')
X=dataset.iloc[:,0:-1].copy()
y=dataset.iloc[:, -1]
```

```
print("Data Before Encoding\n",dataset.head())
print("Features Before Encoding\n",X.head())
print("Target Before Encoding\n",y.head())
```

```
label_encoder={}
```

```
for column in X.columns:
```

```
    le=LabelEncoder()
```

```
    X[column]=le.fit_transform(X[column])
```

```
    label_encoder[column]=le
```

```
print("Features After Encoding\n",X.head())
```

```
target_encoder=LabelEncoder()
```

```
y=target_encoder.fit_transform(y)
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.30)
```

```
model=GaussianNB()
```

```
model.fit(X_train,y_train)
```

```
y_pred=model.predict(X_test)
```

```
print("Actual Label: ",y_test)
```

```
print("Predicted Label: ",y_pred)
```

```
print("Confusion Matrix: \n",confusion_matrix(y_test,y_pred))
```

```
print("Accuracy Score: ",accuracy_score(y_test,y_pred)*100)
```

```
from sklearn.cluster import KMeans

from sklearn.datasets import load_iris

from sklearn.mixture import GaussianMixture

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import adjusted_rand_score

import matplotlib.pyplot as plt

import numpy as np
```

```
dataset = load_iris()

X = dataset.data

y = dataset.target
```

```
plt.figure(figsize=(14,7))

color = np.array(['red', 'lime', 'black'])
```

```
plt.subplot(1,3,1)

plt.scatter(X[:,2], X[:,3], c=color[y], s=40)

plt.title("Real Points")
```

```
kmeans = KMeans(n_clusters=3)

y_kmeans = kmeans.fit_predict(X)
```

```
plt.subplot(1,3,2)
```

```
plt.scatter(X[:,2], X[:,3], c=color[y_kmeans], s=40)

plt.title("K_means")
```

```
mix = GaussianMixture(n_components=3)

X_scaled = StandardScaler().fit_transform(X)

y_mix = mix.fit_predict(X_scaled)
```

```
plt.subplot(1,3,3)

plt.scatter(X[:,2], X[:,3], c=color[y_mix], s=40)

plt.title("Gaussian Mixture")
```

```
print("Performance Score(Kmeans):", adjusted_rand_score(y, y_kmeans))

print("Performance Score(Gaussian Mixture):", adjusted_rand_score(y, y_mix))
```

```
plt.show()
```

```
from sklearn.neighbors import KNeighborsClassifier

from sklearn.datasets import load_iris

from sklearn.metrics import accuracy_score

from sklearn.model_selection import train_test_split
```

```
iris = load_iris()
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```

X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.2
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy Score:", accuracy_score(y_test, y_pred))
print("Actual Value:", iris.target_names[y_test[:10]])
print("Predicted Value:", iris.target_names[y_pred[:10]])

```

```

import matplotlib.pyplot as plt
import numpy as np

```

```

X = np.array([1,2,3,4,5])
y = np.array([1,2,1.2,2.3,3.2])

```

```

x_mat = np.c_[np.ones(len(X)), X]

```

```

def weights(X_query, X, tau):
    return np.exp(-((X - X_query)**2) / (2 * tau**2))

```



```
def lwr(X_query, X, y, tau):  
    wt = weights(X_query, X, tau)  
    W = np.diag(wt)  
    theta = np.linalg.pinv(x_mat.T @ W @ x_mat) @ (x_mat.T @ W @ y)  
    y_pred = theta[0] + theta[1] * X_query  
    return y_pred
```

```
X_query = 3
```

```
tau = 1.0
```

```
y_pred = lwr(X_query, X, y, tau)
```

```
x_val = np.linspace(1, 5, 100)
```

```
y_val = [lwr(x, X, y, tau) for x in x_val]
```

```
plt.plot(x_val, y_val, color='blue', label='LWR')
```

```
plt.scatter(X, y, color='red', label='Data points')
```

```
plt.scatter(X_query, y_pred, color='green', label='Query point')
```

```
plt.legend()
```

```
plt.show()
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    iris.data,  
    iris.target,  
    test_size=0.2,  
    random_state=42  
)
```

```
model = SVC(kernel='linear')
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy Score:", accuracy_score(y_test, y_pred))
```

```
print("\nPredicted Values:", y_pred[:10])
```

```
print("\nActual Values:", y_test[:10])
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
```

```
dataset=load_iris()
```

```
X_train,X_test,y_train,y_test=train_test_split(dataset.data,dataset.target,test_size=0.25,random_state=42)
```

```
model=RandomForestClassifier(n_estimators=100,random_state=42)
```

```
model.fit(X_train,y_train)
```

```
y_pred=model.predict(X_test)
```

```
print("Accuracy Score: ",accuracy_score(y_test,y_pred))
```

```
print("Classification Report: ",classification_report(y_test,y_pred))
```

```
print("Confusion Matrix: ",confusion_matrix(y_test,y_pred))
```

```
import pandas as pd
```

```
import math
```

```
data = pd.read_csv("tennis.csv")
```

```
target = data.columns[-1]
```

```
def entropy(col):
```

```
e = 0
```

```
for v in col.unique():
```

```
    p = sum(col == v) / len(col)
```

```
    e -= p * math.log2(p)
```

```
return e
```

```
def id3(data, attrs):
```

```
    y = data[target]
```

```
    if len(y.unique()) == 1:
```

```
        return y.iloc[0]
```

```
    gains = {}
```

```
    for a in attrs:
```

```
        g = entropy(y)
```

```
        for v in data[a].unique():
```

```
            s = data[data[a] == v]
```

```
            g -= len(s)/len(data) * entropy(s[target])
```

```
        gains[a] = g
```

```
    best = max(gains, key=gains.get)
```

```
    tree = {best: {}}
```

```
    for v in data[best].unique():
```

```
        s = data[data[best] == v]
```

```
        if len(s) == 0:
```

```
            tree[best][v] = y.mode()[0]
```

else:

```
tree[best][v] = id3(  
    s.drop(columns=[best]),  
    [x for x in attrs if x != best]  
)
```

return tree

def print_tree(tree, space=""):

if not isinstance(tree, dict):

print(space + str(tree))

return

for k, v in tree.items():

print(space + k)

for val, child in v.items():

print(space + " " + str(val))

print_tree(child, space + " ")

tree = id3(data, list(data.columns[:-1]))

print_tree(tree)